

Exercise 1 - Introduction to Simulation with Variance Estimation

Florescu Andrei - 12242987

2024

Setup

Set seed

```
set.seed(12242987)
```

Datasets

```
x1 <- rnorm(100)
x2 <- rnorm(100, mean = 1000000)
```

1. Algorithm function implementations

Algorithm 1 - (two - pass algorithm - variance calculation in R)

```
precise_var <- function(data) {
  sample_mean <- mean(data)
  sum <- sum((data - sample_mean)^2)
  variance <- sum / (length(data) - 1)
  return(variance)
}
```

Algorithm 2 - (one - pass algorithm - previously variance calculation in Excel)

```
excel_var <- function(data) {
  p1 <- sum(data^2)
  p2 <- (sum(data)^2) / length(data)
  variance <- (p1 - p2) / (length(data) - 1)
  return(variance)
}
```

Algorithm 3 - (shifted one - pass algorithm):

```
shifted_var <- function(data, c) {
  p1 <- sum((data - c)^2)
  p2 <- sum(data - c)^2 / length(data)
  variance <- (p1 - p2) / (length(data) - 1)
  return(variance)
}
```

Algorithm 4 - (online):

```
online_var <- function(data) {  
  sample_mean <- mean(data[1:2])  
  sum <- sum((data[1:2] - sample_mean)^2)  
  variance <- sum / (length(data[1:2]) - 1)  
  
  for (i in 3:length(data)) {  
    sample_mean1 <- sample_mean  
    sample_mean <- sample_mean + (data[i] - sample_mean) / i  
    variance <- (i - 2) * variance / (i - 1) + ((data[i] - sample_mean1)^2) / i  
  }  
  
  return(variance)  
}
```

Algorithm function wrapper

```
compute_variance <- function(data, type) {  
  result <- switch(type,  
    "precise" = precise_var(data),  
    "excel" = excel_var(data),  
    "shifted" = shifted_var(data, data[1]),  
    "online" = online_var(data),  
    "var" = var(data)  
  )  
  return(result)  
}
```

Call each compute variance

```
precise1 <- compute_variance(x1, "precise")  
precise2 <- compute_variance(x2, "precise")  
excel1 <- compute_variance(x1, "excel")  
excel2 <- compute_variance(x2, "excel")  
shifted1 <- compute_variance(x1, "shifted")  
shifted2 <- compute_variance(x2, "shifted")  
online1 <- compute_variance(x1, "online")  
online2 <- compute_variance(x2, "online")  
rvar1 <- compute_variance(x1, "var")  
rvar2 <- compute_variance(x2, "var")
```

Variance comparison using all.equal

```
all.equal(precise1, excel1, shifted1, online1, rvar1)
```

```
## [1] TRUE
```

```
all.equal(precise2, excel2, shifted2, online2, rvar2)
```

```
## [1] TRUE
```

Variance comparison for x1 using ==

```
variables1 <- c(precise1, excel1, shifted1, online1)
names <- c("precise", "excel", "shifted", "online")
comparison_matrix1 <- matrix(variables1 == rvar1, nrow = 1)
colnames(comparison_matrix1) <- names
rownames(comparison_matrix1) <- "R var function"
print(comparison_matrix1)

##           precise excel shifted online
## R var function  FALSE FALSE   TRUE  FALSE
```

Variance comparison for x2 using ==

```
variables2 <- c(precise2, excel2, shifted2, online2)
comparison_matrix2 <- matrix(variables2 == rvar2, nrow = 1)
colnames(comparison_matrix2) <- names
rownames(comparison_matrix2) <- "R var function"
print(comparison_matrix2)

##           precise excel shifted online
## R var function   TRUE  FALSE   TRUE  FALSE
```

Variance comparison for x1 using identical

```
names <- c("precise", "excel", "shifted", "online")
comparison_matrix1 <- matrix(sapply(variables1, function(x) identical(x, rvar1)), nrow = 1)
colnames(comparison_matrix1) <- names
rownames(comparison_matrix1) <- "R var function"
print(comparison_matrix1)

##           precise excel shifted online
## R var function  FALSE FALSE   TRUE  FALSE
```

Variance comparison for x2 using identical

```
comparison_matrix2 <- matrix(sapply(variables2, function(x) identical(x, rvar2)), nrow = 1)
colnames(comparison_matrix2) <- names
rownames(comparison_matrix2) <- "R var function"
print(comparison_matrix2)

##           precise excel shifted online
## R var function   TRUE  FALSE   TRUE  FALSE
```

We can see that using == and identical yields the same results for comparing these variances. We can also see that all.equal returned TRUE for both datasets even though this is not right. Therefore we can conclude that it's more feasible to use == operator or the identical function to compare results. Another thing to note is that the only algorithm that returned the same result as the R var function is the shifted algorithm. The precise algorithm returned the same value only once, and the rest didn't return the same exact result in either case.

```
variables1 <- c(variables1, rvar1)
variables2 <- c(variables2, rvar2)
names <- c(names, "R var")
result <- matrix(c(variables1, variables2), nrow = 2)
```

```
colnames(result) <- names
rownames(result) <- c("mean=0", "mean=1000000")
print(result)
```

```
##           precise    excel  shifted  online    R var
## mean=0      1.121355 1.121355 1.121355 1.114741 1.11463
## mean=1000000 1.121355 1.121355 1.114630 1.114630 1.11463
```

2. Compare computational performance

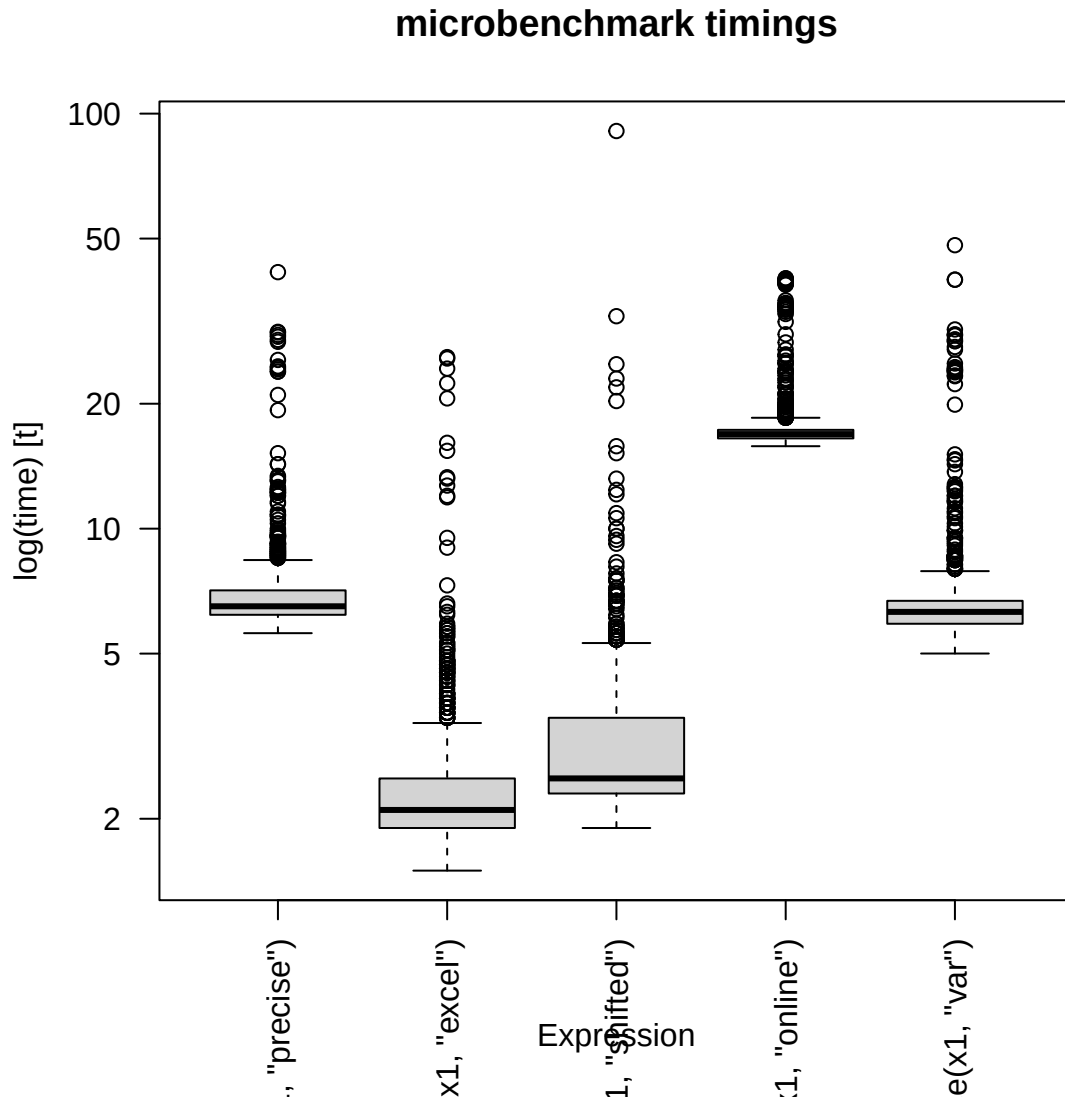
```
library(microbenchmark)
```

```
benchmark_result <- microbenchmark(
  compute_variance(x1, "precise"),
  compute_variance(x1, "excel"),
  compute_variance(x1, "shifted"),
  compute_variance(x1, "online"),
  compute_variance(x1, "var"),
  times = 1000L
)
```

```
print(benchmark_result)
```

```
## Unit: microseconds
##           expr    min      lq      mean median      uq      max
## compute_variance(x1, "precise") 5.600  6.201  7.134204  6.501  7.101 41.500
##   compute_variance(x1, "excel") 1.500  1.900  2.555487  2.100  2.501 25.901
##   compute_variance(x1, "shifted") 1.900  2.301  3.307909  2.502  3.502 90.801
##   compute_variance(x1, "online") 15.800 16.501 17.683313 16.900 17.301 40.101
##   compute_variance(x1, "var")   5.002  5.900  6.998079  6.302  6.701 48.201
## neval
## 1000
## 1000
## 1000
## 1000
## 1000
```

```
par(las = 2)
boxplot(benchmark_result)
```



From the boxplot we can see that the slowest algorithm is the online one while the fastest algorithm is the excel one. We can also see that the times are closer to each other for precise and online while for the rest they have more scattered runtimes.

3. Scale invariance property

```
variances <- c(
  shifted_var(x1, x1[1]),
  shifted_var(x1, 1),
  shifted_var(x1, 100000),
  shifted_var(x1, mean(x1)),
  var(x1)
)
comparison_matrix <- matrix(variances, nrow = 1)
names <- c("x1[1]", "1", "1000", "mean", "R var function")
colnames(comparison_matrix) <- names
```

```
rownames(comparison_matrix) <- "Variance of x1"
print(comparison_matrix)
```

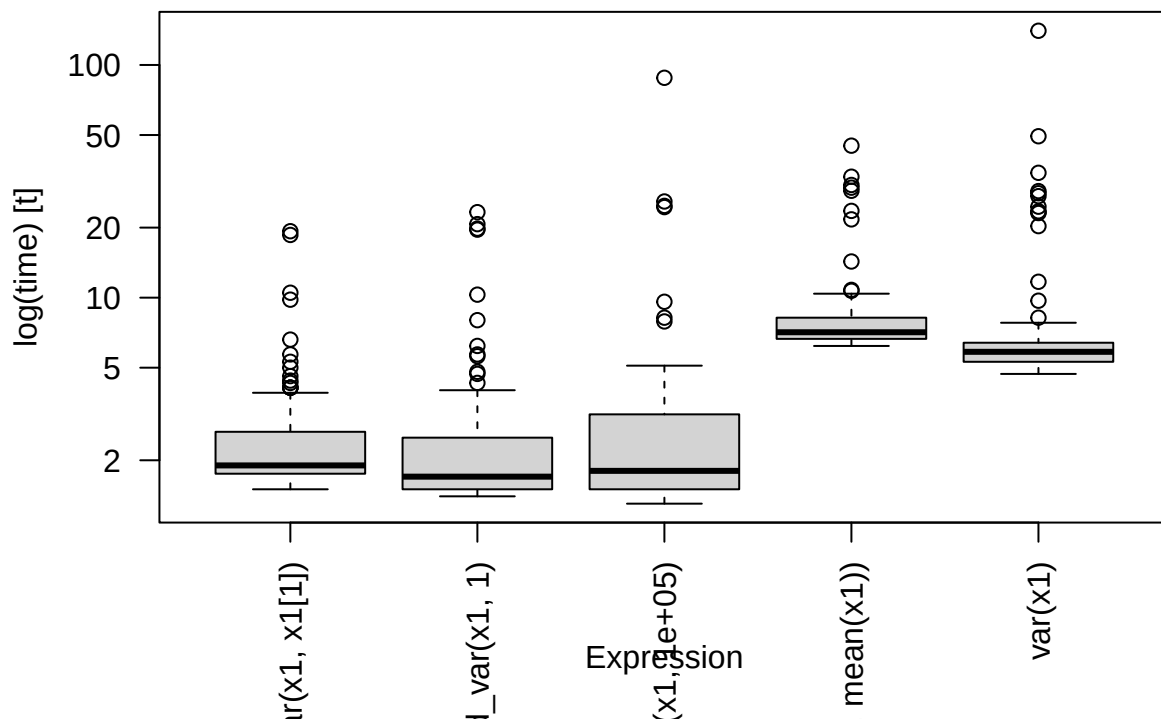
```
##              x1[1]          1      1000      mean R var function
## Variance of x1 1.121355 1.121355 1.121354 1.121355      1.121355
```

```
benchmark_result <- microbenchmark(
  shifted_var(x1, x1[1]),
  shifted_var(x1, 1),
  shifted_var(x1, 100000),
  shifted_var(x1, mean(x1)),
  var(x1)
)
print(benchmark_result)
```

```
## Unit: microseconds
##              expr    min      lq    mean median      uq      max neval
##  shifted_var(x1, x1[1]) 1.501  1.751  2.83505  1.9010  2.6510  19.300   100
##  shifted_var(x1, 1)    1.400  1.501  3.01391  1.7010  2.5015  23.301   100
##  shifted_var(x1, 1e+05) 1.302  1.501  3.89297  1.8010  3.1510  88.101   100
##  shifted_var(x1, mean(x1)) 6.200  6.651  9.02789  7.1005  8.2010  45.002   100
##              var(x1) 4.701  5.301  9.43400  5.8510  6.4010 140.302   100
```

```
par(las = 2)
boxplot(benchmark_result)
```

microbenchmark timings



4. Condition numbers

```
calculate_condition_number <- function(data) {  
  S <- sum((data - mean(data))^2)  
  k <- sqrt(1 + (mean(data)^2) * length(data) / S)  
  return(k)  
}  
  
calculate_condition_number_shifted <- function(data, c) {  
  S <- sum((data - mean(data))^2)  
  k <- sqrt(1 + ((mean(data) - c)^2) * length(data) / S)  
  return(k)  
}  
  
calculate_condition_number(x1)  
  
## [1] 1.031974  
calculate_condition_number(x2)  
  
## [1] 951956.3  
calculate_condition_number_shifted(x1, mean(x1))  
  
## [1] 1  
calculate_condition_number_shifted(x2, mean(x1))  
  
## [1] 951956.1  
x3 <- sample(1:1000000, 100, replace = TRUE)  
calculate_condition_number(x3)  
  
## [1] 2.143626  
calculate_condition_number_shifted(x3, 0.1)  
  
## [1] 2.143625
```